

DIMSE PACS E2E Validation Runbook

Created: 2026-02-20 Branch target: `feature/dimse-pacs-e2e-validation`

Goal

Validate enterprise DIMSE ingress reliability from C-STORE send through ingest retry/dead-letter controls with reproducible operator actions.

Harness

Prerequisites:

```
cd dimse-receiver && pip install -r requirements.txt -r requirements-test.txt
```

Command:

```
python3 scripts/dimse_pacs_e2e_harness.py
```

Optional flags:

```
python3 scripts/dimse_pacs_e2e_harness.py \  
  --timeout 45 \  
  --dimse-ae-title AEGIS \  
  --workdir dimse-receiver \  
  --keep-logs
```

What it starts automatically:

- a mock ingest API (`POST /api/ingest`) that can be toggled success/fail
- `dimse-receiver` via `uvicorn app.main:app` on ephemeral HTTP + DICOM ports
- synthetic C-STORE sender using `pynetdicom` + generated DICOM datasets

Scenario Coverage

The harness runs these scenarios in order and exits non-zero on first failure:

1. `success_c_store_ingest`
 - sends study via C-STORE
 - expects ingest success and retry snapshot `pending=0` , `dead_letter=0`
2. `transient_failure_to_retry_queue`
 - forces ingest API failure
 - sends study via C-STORE
 - expects study in `GET /ingest/retry/details?study_instance_uid=...` pending queue
3. `process_controls_restore_ingestion`
 - starts from pending retry state
 - restores ingest API success
 - runs `POST /ingest/retry/process/{study_instance_uid}`

- expects result `ok` and study cleared from retry/dead-letter

4. `dead_letter_path_and_recovery`

- forces repeated failures until dead-letter
- validates `dead_letter_nonzero` appears in `GET /healthz` degraded reasons
- runs `POST /ingest/retry/replay/{study_instance_uid}` then targeted process
- expects successful recovery and cleared retry/dead-letter state

Key Endpoints and Expected Fields

`GET /healthz :`

- `status` : `ok` or `degraded`
- `degraded_reasons` : includes `dead_letter_nonzero` when dead-letter queue is non-empty
- `ingest_retry` : includes `pending` , `dead_letter` , age counters, and next-attempt timing

`GET /ingest/retry :`

- `status` : `ok`
- `ingest_retry.pending` , `ingest_retry.dead_letter`

`GET /ingest/retry/details?study_instance_uid=<uid> :`

- `ingest_retry.pending_total`
- `ingest_retry.dead_letter_total`
- `ingest_retry.pending_items[]` / `dead_letter_items[]`

`POST /ingest/retry/process/{study_instance_uid} :`

- `ingest_retry.found`
- `ingest_retry.result` : one of `ok` , `requeued` , `dead_letter` , `not_found`

`POST /ingest/retry/replay/{study_instance_uid} :`

- `ingest_retry.found`
- `ingest_retry.moved`
- queue before/after counters

Evidence Capture

Harness output includes per-scenario pass/fail with timing and a final summary. On failure, the script prints a tail of `dimse-receiver` logs for diagnostics.

Recommended evidence artifacts per run:

- full harness stdout
- failing run log tail (if any)
- date/time and git commit SHA